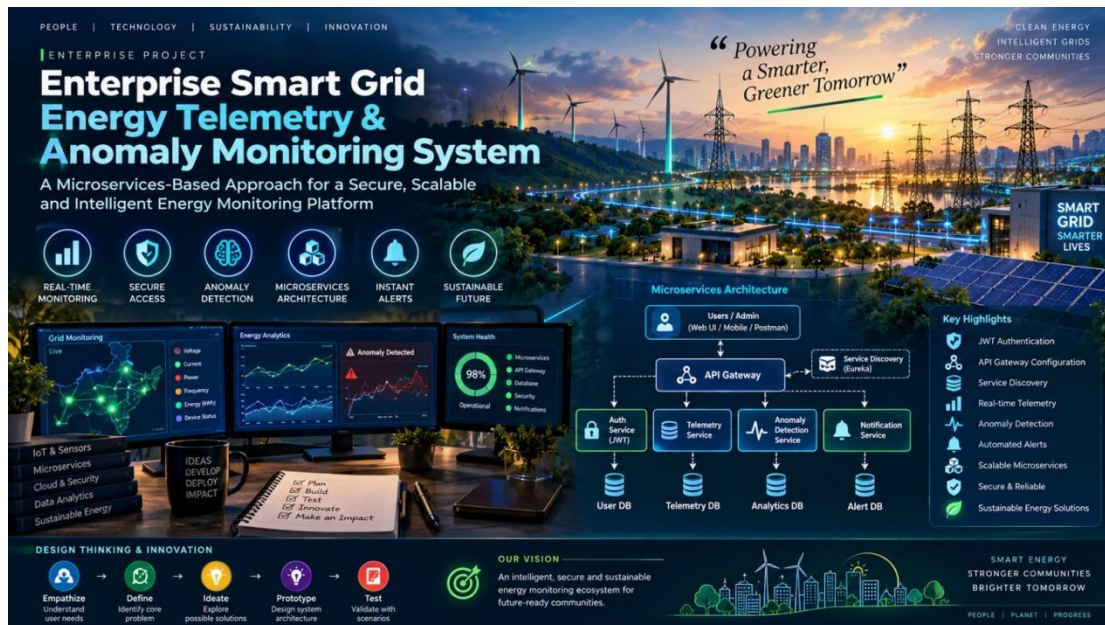


Enterprise Smart Grid Energy Telemetry & Anomaly Monitoring System



September 21, 2026

🔗 A Microservices-Based Approach to Smart Energy Monitoring and Consumption Management

Project: Smart Energy Consumption Monitoring System – Project 41

Domain: Smart Energy Management

Project Mentor: Dr. Sindhe Phani Kumar

Institution: KL University

Department: Computer Science and Engineering

🌐 Introduction:

Energy consumption monitoring plays an important role in understanding how energy is being utilized and identifying situations where consumption increases beyond expected levels. With the increasing need for efficient energy management, there is a requirement

for systems that can monitor energy usage and provide timely information about high consumption.

Our project, Smart Energy Consumption Monitoring System, focuses on developing a secure and structured platform for monitoring energy consumption, managing smart meters, and identifying threshold violations.

The system follows a microservices architecture, where authentication, meter management, usage monitoring, and alert management are developed as independent services.

The project combines Spring Boot, Spring Cloud, JWT Authentication, API Gateway, Eureka Service Discovery, PostgreSQL, REST APIs, and Load Balancing to create an organized and scalable solution.

Problem Analysis and Requirement Specification

During the problem analysis phase, we focused on understanding the challenges involved in monitoring energy consumption.

Users may find it difficult to continuously track their energy consumption and identify situations where consumption increases beyond an expected level. Managing information from multiple meters and maintaining consumption records can also become difficult.

Based on our analysis, we identified the following major requirements:

-  Secure authentication for users
-  Smart meter information management
-  Energy consumption recording and monitoring
-  Threshold-based consumption monitoring
-  Alert generation when consumption exceeds a configured threshold
-  Centralized access to different services
-  Service discovery between microservices
-  Secure access to protected APIs
-  Integration between usage monitoring and alert management

Based on these requirements, we selected a microservices-based architecture to divide the system into independent and manageable components.

Survey Conducted and Common Issue Addressed

As part of our problem understanding, we focused on the difficulties users may face while monitoring their energy consumption.

The survey activity considered how users monitor their energy usage, how they identify high consumption, and how they become aware when their consumption crosses an expected limit.

A common issue identified was the difficulty of continuously monitoring energy consumption manually.

Users need a simple way to understand their consumption and identify situations where energy usage exceeds a predefined threshold.

Based on this understanding, our project focuses on connecting energy usage monitoring with threshold-based alert management.

Design Thinking Approach

Design Thinking was used to understand the problem from the user's perspective and translate the identified needs into a practical technical solution.

Empathy Mapping

The empathy mapping process helped us understand the thoughts, feelings, actions, and expectations of an energy consumer.

Users want to know how much energy they are consuming and want to identify high-consumption situations.

They may think about why their energy consumption has increased and whether they can identify unusual usage quickly.

Users generally monitor their consumption information and meter details, while high consumption can create concern.

This helped us identify the need for a system that provides organized consumption information and automatic threshold checking.

User Persona

The user persona represents an energy consumer who wants to monitor energy usage and understand when consumption becomes higher than the expected level.

Goals

-  Monitor energy consumption
-  Understand usage information
-  Identify high consumption
-  Monitor meter information
-  Become aware of threshold violations

Pain Points

Difficulty in continuous monitoring
Lack of immediate awareness of high consumption
Managing consumption information from multiple meters
Manual identification of threshold violations

Expectations

The user expects a system that is secure, simple, organized, and capable of identifying threshold violations automatically.

Customer Journey Map

The customer journey in our system begins with user authentication and continues through meter monitoring, usage recording, threshold checking, and alert management.

Login → Authentication → Meter Monitoring → Usage Recording → Threshold Checking → Alert Generation → Review

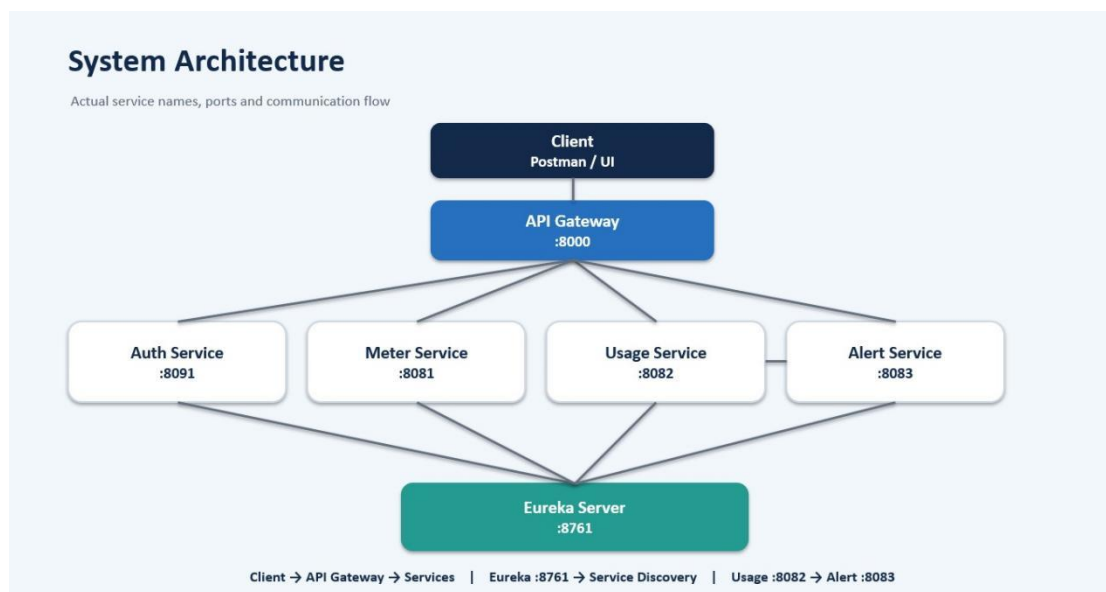
This journey helped us understand how different user needs could be connected through the functionalities of the system.



System Architecture

The proposed Smart Energy Consumption Monitoring System follows a microservices-based architecture designed to separate the major functionalities of the application into independent services.

The architecture consists of an API Gateway, Authentication Service, Meter Service, Usage Service, Alert Service, and Eureka Server.



The API Gateway acts as the single entry point for client requests and routes requests to the appropriate microservice.

The Authentication Service manages user authentication and JWT generation.

The Meter Service manages smart meter information, while the Usage Service manages energy consumption records.

The Alert Service handles threshold-based monitoring and alert management.

The Eureka Server provides service discovery between the microservices.

This architecture provides clear separation of responsibilities and makes the system easier to manage and extend.

Microservices

Authentication Service

The Authentication Service is responsible for user authentication and authorization.

It manages user registration and login and generates JWT tokens after successful authentication. The generated token is used to access protected APIs through the system.

This service provides the security layer while keeping authentication functionality separate from the other services.

Meter Service

The Meter Service manages smart meter information.

It maintains information related to meters and their association with users.

The service provides the required operations for managing meter information and supports organized management of different meters within the system.

Usage Service

The Usage Service manages energy consumption information.

It records consumption associated with individual meters along with the corresponding timestamp.

This service forms an important part of the monitoring workflow by maintaining energy usage information.

The Usage Service also communicates with the Alert Service when new consumption information is recorded.

Alert Service

The Alert Service manages threshold-based alerts.

It maintains threshold information associated with meters and checks incoming consumption against the configured threshold.

When consumption reaches or exceeds the defined threshold, the corresponding alert status is updated.

This allows the system to identify high-consumption situations through an automated threshold-checking process.

Eureka Server

The Eureka Server provides service discovery for the microservices.

Each service registers itself with Eureka, allowing the services to discover and communicate with each other within the application.

This reduces dependency on fixed service locations and supports the distributed architecture.

API Gateway

The API Gateway acts as the centralized entry point for client requests.

It receives client requests and routes them to the appropriate microservice.

It also works with JWT authentication to provide secure access to protected services.

Data Handling and Service Integration

Each microservice is responsible for managing the data related to its specific functionality.

The Authentication Service handles authentication-related information, the Meter Service manages meter information, the Usage Service manages energy consumption records, and the Alert Service manages threshold and alert information.

The major service integration takes place between the Usage Service and Alert Service.

When new consumption information is recorded through the Usage Service, it communicates with the Alert Service for threshold evaluation.

The Alert Service evaluates the consumption against the configured threshold and updates the alert status when the threshold condition is reached.

This integration connects energy usage monitoring with automated alert management and demonstrates communication between independent microservices.

JWT Authentication

Security is an important part of our system.

We implemented JWT-based authentication to secure access to protected APIs.

The authentication process begins when the user provides login credentials to the Authentication Service.

After successful authentication, a JWT token is generated and used for accessing protected APIs through the API Gateway.

This approach provides secure access while keeping authentication separate from the business functionalities of the application.

Service Discovery and Load Balancing

The system uses Eureka Service Discovery to allow microservices to register and discover one another.

Spring Cloud LoadBalancer is also integrated into the architecture to support service-level load balancing.

This approach provides a foundation for handling multiple instances of services when the application needs to scale.

It also reduces dependency on fixed service locations and supports a flexible microservices environment.

Technologies Used

The project was developed using modern technologies for backend and microservices development.

Java was used as the primary programming language, with Spring Boot for developing individual services.

Spring Cloud was used for service discovery, API Gateway functionality, and load balancing.

PostgreSQL was used for data management, while Spring Data JPA and Hibernate were used for database interaction.

JWT was used for authentication and REST APIs were used for communication between services.

Maven was used for project management and Postman was used during API validation.

Innovative Ideas

The developed system provides a foundation for several future improvements in smart energy management.

Energy Consumption Dashboard

A dashboard can provide a visual representation of energy consumption and help users understand their usage patterns.

Personalized Thresholds

Different threshold values can be configured for individual users or meters based on their consumption requirements.

Consumption Trend Analysis

Historical consumption information can be analyzed to identify energy usage patterns.

Notification Mechanism

Future versions can integrate email, SMS, or mobile notifications to inform users about important threshold violations.

Predictive Energy Analysis

Historical consumption data can be used in future versions for predictive analysis and improved energy management.

Cloud Deployment

The microservices can be containerized and deployed on cloud infrastructure to support scalable deployment.

Project Outcome

The project helped us transform a real-world energy monitoring problem into a structured software solution.

By combining Design Thinking with Microservices Architecture, we were able to understand user requirements and translate them into independent functional services.

The developed system brings together:

 Secure authentication  Smart meter management  Energy usage monitoring  Threshold-based alert management  Service discovery  API Gateway  Inter-service communication  Load-balancing support

The project provided practical experience in designing and developing a distributed application using modern software technologies.

Conclusion

The Smart Energy Consumption Monitoring System demonstrates how a real-world energy monitoring problem can be addressed through a combination of Design Thinking and Microservices Architecture.

The Design Thinking approach helped us understand user needs through Empathy Mapping, User Persona, and Customer Journey Mapping.

The technical architecture translated these requirements into independent services for authentication, meter management, energy usage management, and alert management.

By integrating Spring Boot, Spring Cloud, JWT, Eureka, API Gateway, PostgreSQL, REST APIs, and Load Balancing, the project provides a structured foundation for smart energy monitoring.

This project helped our team gain practical experience in problem analysis, user-centered design, microservices development, secure API design, service discovery, and inter-service communication.

The system also provides a foundation for future development toward more advanced and intelligent energy management solutions.



Team Details

Smart Energy Consumption Monitoring System – Project 41

Institution: KL University Department: Computer Science and Engineering

Team Lead: Kotini Jeswanth[2400032703]

Team Members:

2400032703 - Kotini Jeswanth

2400030053 - Praneeth Sriram Desimsetti

2400030040 - Nelavelli Sri Aravind



Technologies

Java | Spring Boot | Spring Cloud | Microservices | JWT | Eureka | API Gateway | PostgreSQL | Hibernate | REST APIs | Maven | Postman



Hashtags

#SmartEnergy #EnergyMonitoring #Microservices #SpringBoot #Java #SpringCloud #JWT #APIGateway #Eureka #PostgreSQL #RESTAPI #DesignThinking #SoftwareEngineering #KLUniversity #StudentProject #ContinuousEvaluationrsity #StudentProject

Original

LinkedInArticle:<https://www.linkedin.com/pulse/enterprise-smart-grid-energy-telemetry-anomaly-system-jeswanth-kotini-occuf/?trackingId=aV8cWdAJQMKrHoWuCEI80A%3D%3D>

